

SOC MVP and decision-making support system

Realization report

Artur Kraskov
ICT & OL, Delta
Semester 6
Fontys 2025

Contents

Contents.....	1
Introduction.....	2
Background.....	2
Realization.....	2
1. Demo.....	2
2. Repository Structure.....	3
3. System Architecture and Components.....	4
3.1 Core Components.....	4
3.2 Code snippets.....	4
Apple Metal Acceleration for Graph Algorithm.....	4
SSH data streaming.....	7
MITRE LookUp & Mitigations query.....	8
SSH tunnel to enable Caldera locally.....	8
APT group MITRE techniques ingestion with Kill Chain mapping.....	9
Caldera APT auto launcher.....	9
3.3 User Interface.....	10
4. Configuration and Change Management.....	11
4.1 Infrastructure Management.....	11
4.2 Configuration Management.....	11
4.3 Release Management.....	12
Conclusion.....	12
Reference.....	12

Introduction

Within the scope of MVP Security Operations Center project a research was conducted into analytical and cybersecurity frameworks, tools and best practices. Based on literature study a minimal system configuration and software design were compiled. An attempt to realize intuitive and visual decision making support tool was performed. This document covers the realization process, provides the demo. Information about configuration, change and knowledge transfer is included.

Background

Previous work includes making a project proposal, conducting research, analysing, designing and realizing infrastructure for this project [1-2]. New updated Security Operations Center plan and software architecture were developed after the research was finished [3-4]. Current realization is based on updated data.

Realization

The SOC Dashboard Visualizer is a sophisticated security operations center visualization tool built using Streamlit. It serves as a proof-of-concept for educational purposes, demonstrating the integration of various security tools and providing real-time visualization of security alerts. The system is designed to be scalable and integrates with existing security frameworks while maintaining a modular architecture.

1. Demo

Part 1: <https://youtu.be/Su0V5RITrwM>

Part 2: <https://youtu.be/rmcvJM2Erw>

Github: https://github.com/Krasnomakov/soc_mvp/tree/main

2. Repository Structure

The repository of the application has the following structure:

```
my_app/                                # Main application folder
├── app.py                             # Main application entry point
├── common.py                           # Shared utilities and functions
├── um_gen.py                           # Universal machine generator
├── um_gen_metal.py                     # Optimized version for Apple Silicon/Intel Macs
├── requirements.txt                     # Python dependencies
├── case_data.csv                       # Sample data for analysis
├── Figure_1.png                        # Network diagram
├── __pycache__/                        # Python bytecode cache
├── pages/                              # Streamlit pages for different features
│   ├── getting_started_page.py         # Complete SOC set up guide, 12 steps
│   ├── wazuh_plotly_js_four_graphs.py  # Alerts Triage Graphs
│   ├── wazuh_plotly_template.html      # HTML and JS frontend for interactive graph
│   ├── wazuh_plotly_js_interactive.py  # Interactive graph backend
│   ├── chat_page.py                   # LLM agent & MITRE query, case management
│   ├── machine_traffic.py              # Network traffic monitoring page
│   ├── snort.py                        # Snort Alerts streaming
│   ├── red_blue_emulator.py            # Caldera over SSH tunnel
│   └── wazuh_plotly_js.py              # Graph and alerts stream (basic)
├── mitre_querying/                    # MITRE ATT&CK integration
│   ├── lookup_and_mitigation.py        # Get threat data and mitigation from MITRE
│   ├── mitre_lookup.py                 # Only get threat data (no mitigations)
│   └── enterprise-attack.json           # Local MITRE ATT&CK ontology
├── images/                            # Documentation and UI assets
│   ├── step_1_abstract_network.png     # Image for getting started guide
│   ├── step_2_wazuh_in_docker.png     # Image for getting started guide
│   ├── step_3_wazuh_agent_sysmon.png   # Image for getting started guide
│   └── ...                             # Remaining images omitted
├── .streamlit/                         # Streamlit configuration hidden folder
└── pages.toml                          # Streamlit pages configuration file
```

3. System Architecture and Components

3.1 Core Components

Component	Description
Main Application (app.py)	• Implements the primary Streamlit interface • Handles user authentication and session management • Provides navigation between different dashboard components
Security Tool Integration	• Wazuh SIEM integration for log alerting and log management • Snort IDS for network traffic monitoring • MITRE Caldera for threat emulation • MITRE ATT&CK for threat intelligence
Pages	• Interactive plots for alert analysis • Real-time network traffic monitoring • Rule-level graph visualization (Triage) • Snort IDS alerts • Threat Intelligence (MITRE & AI) • Getting Started guide
AI	• Ollama server • Open-source general purpose LLM and reasoning models
Graph generation	• Custom algorithm for universal matrix graph generation • Accelerated version to speed up computation on Mac with Apple Metal (relevant to deep graphs with 10-30K nodes)

3.2 Code snippets

Several code snippets are provided covering main back end functionalities.

Apple Metal Acceleration for Graph Algorithm

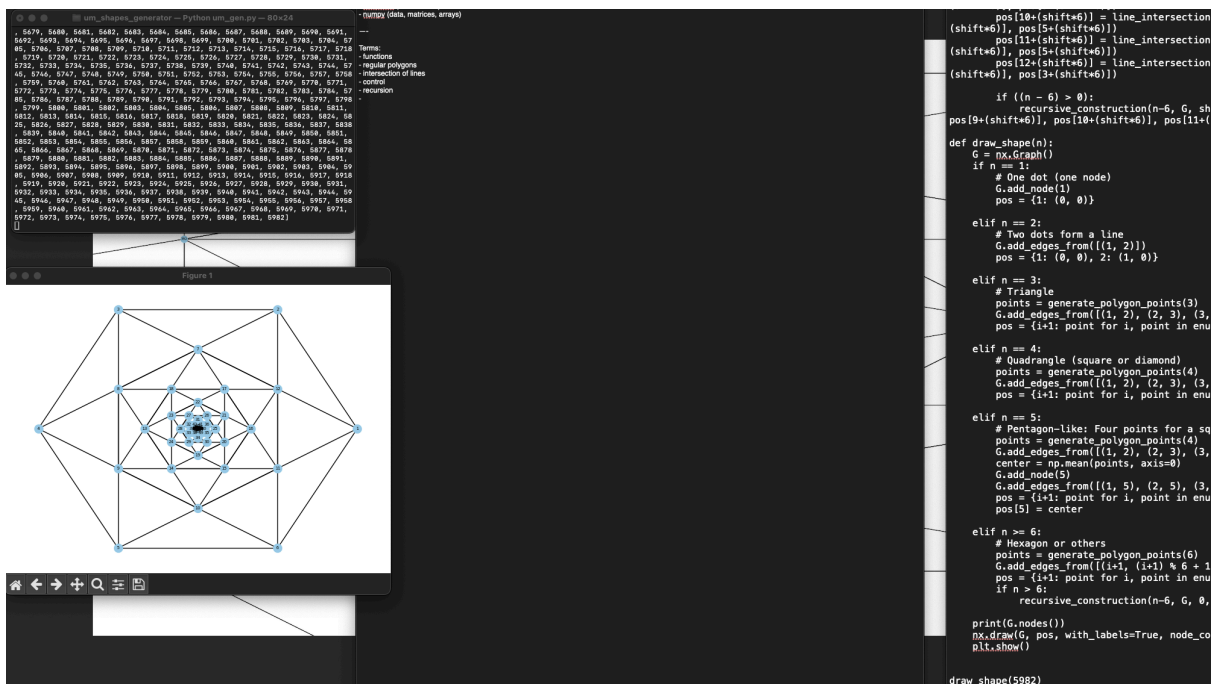
One of particularly notable code contributions is in graph accelerator. Upon testing the initial algorithm capabilities to generate graphs with large numbers of nodes the limit was found at 5982 nodes. 5983 was already too much. Running code was exiting without an image produced. After implementing this upgrade the algorithm running locally on MacBook Pro with M1 chip became capable of generating graphs with up to 30000 nodes. This code upgrade is relevant for live alert streaming with a bigger network. It must be fast and allow the dashboard to generate deep graphs. Original code is based on research of the universal matrix and is provided by the author of the research paper [5]. Code is not public.

```

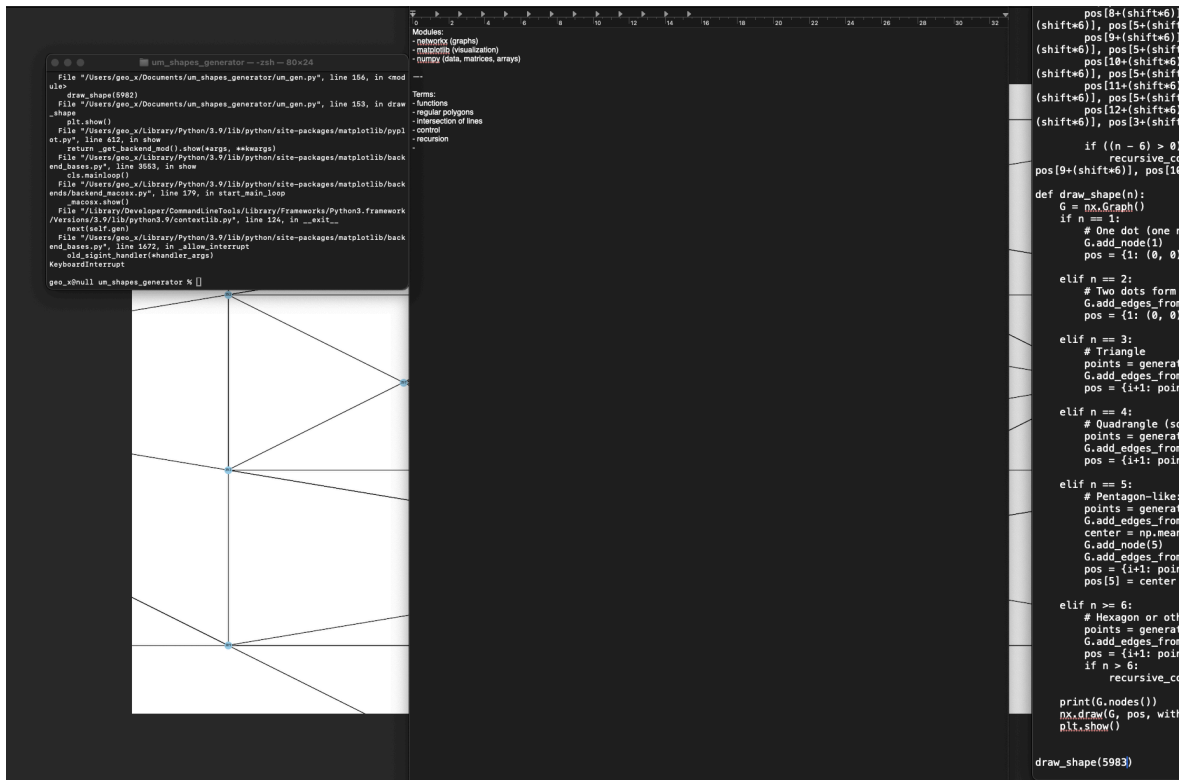
um_gen_metal.py > ...
1  import sys
2  import networkx as nx
3  import matplotlib.pyplot as plt
4  import numpy as np
5  import torch
6  import math
7
8  # Raise recursion limit so large n won't exceed it
9  sys.setrecursionlimit(50000)
10
11 # Choose MPS if available, else CPU fallback
12 device = torch.device("mps") if torch.backends.mps.is_available() else torch.device("cpu")
13
14 # Updated function: generate polygon points on GPU using PyTorch and return them as a NumPy array
15 def generate_polygon_points(n, radius=1):
16     # Create angles on GPU using arange
17     angles = torch.arange(0, 2*math.pi, (2*math.pi)/n, device=device, dtype=torch.float32)
18
19     # In case float rounding yields n+1 elements, just trim the extras
20     if angles.shape[0] > n:
21         angles = angles[:n]
22
23     x = radius * torch.cos(angles)
24     y = radius * torch.sin(angles)
25
26     # Stack into (x,y) points
27     points = torch.stack([x, y], dim=1)
28     # Move back to CPU as a NumPy array for NetworkX/Matplotlib
29     return points.cpu().numpy()
30
31 # Updated function: do line-intersection math on GPU
32 def line_intersection(p1, p2, p3, p4):
33     # Convert to torch tensors on the chosen device
34     p1 = torch.tensor(p1, dtype=torch.float32, device=device)
35     p2 = torch.tensor(p2, dtype=torch.float32, device=device)
36     p3 = torch.tensor(p3, dtype=torch.float32, device=device)
37     p4 = torch.tensor(p4, dtype=torch.float32, device=device)
38
39     denom = (p1[0] - p2[0])*(p3[1] - p4[1]) - (p1[1] - p2[1])*(p3[0] - p4[0])
40
41     # Move denom back to CPU for a zero check
42     if denom.item() == 0:
43         return None # parallel or coincident lines
44
45     intersect_x = (
46         (p1[0]*p2[1] - p1[1]*p2[0])*(p3[0] - p4[0])
47         - (p1[0] - p2[0])*(p3[0]*p4[1] - p3[1]*p4[0])
48     ) / denom
49     intersect_y = (
50         (p1[0]*p2[1] - p1[1]*p2[0])*(p3[1] - p4[1])
51         - (p1[1] - p2[1])*(p3[0]*p4[1] - p3[1]*p4[0])
52     ) / denom
53
54     return (float(intersect_x.item()), float(intersect_y.item()))

```

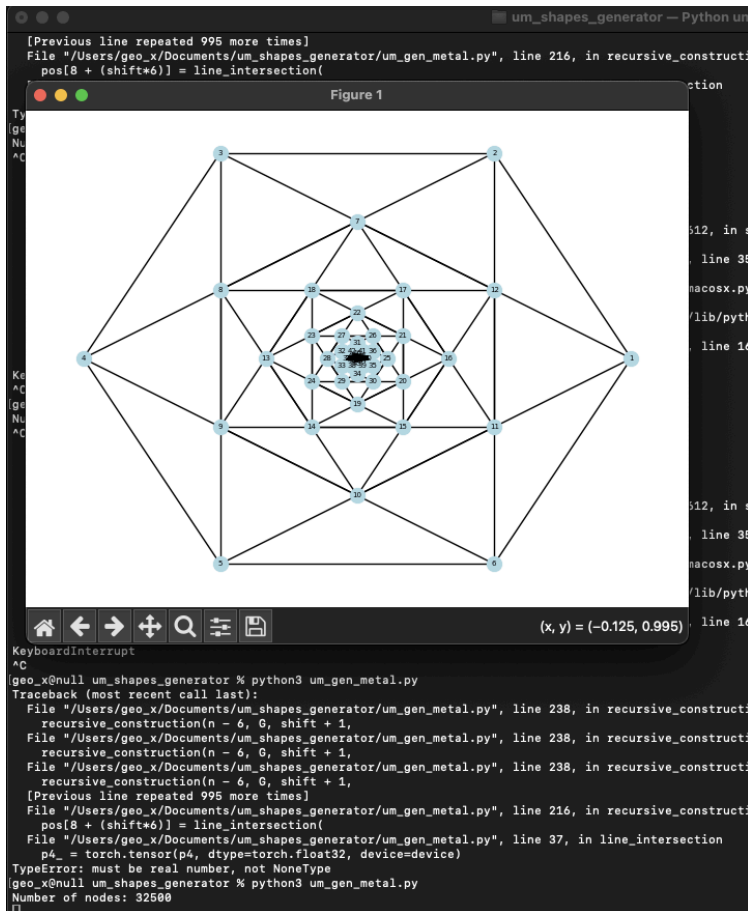
Acceleration of graph generator with torch (offloading to GPU and returning to CPU on M1 chip)



5982 node graph generated



5983 nodes graph generation failed



Successful generation of the graph with 32500 nodes using Apple Metal

SSH data streaming

The dashboard receives data from a remote host. Python functions were implemented to allow this functionality in common.py file. The file can be called by multiple pages and use Wazuh alerts stream for interactive or triage graphs.

```
226 def ssh_stream_wazuh(user: str, ip: str, pw: str, q: queue.Queue):
227     client: paramiko.SSHClient | None = None
228     try:
229         client = open_ssh_client(ip, user, pw)
230         cmd = (
231             f"sudo -S tail -n 1000 -F {_WAZUH_DOCKER_ALERTS_JSON}"
232         )
233         stdin, stdout, _ = client.exec_command(cmd)
234         stdin.write(pw + "\n")
235         stdin.flush()
236         for line in stdout:
237             q.put(line.strip())
238     except Exception as e: # pragma: no cover - bubble up error via queue
239         q.put(json.dumps({"error": str(e)}))
240     finally:
241         if client is not None:
242             client.close()
243
```

common.py ssh_stream_wazuh function

```
245 def ssh_stream_snort(
246     user: str,
247     ip: str,
248     pw: str,
249     host: str,
250     q: queue.Queue,
251 ):
252     client: paramiko.SSHClient | None = None
253     try:
254         client = open_ssh_client(ip, user, pw)
255         # Base Snort command
256         base_cmd = "sudo -S snort -A console -c /etc/snort/snort.conf -i ens192 host "
257
258         # Append host filter if provided. The 'host' parameter is expected to be
259         # a valid BPF filter string like "host 192.168.1.1" or "net 192.168.0.0/16 or port 80"
260         # If host is empty, no filter is applied.
261         cmd = f"{base_cmd} {host}" if host else base_cmd
262
263         stdin, stdout, stderr = client.exec_command(cmd, get_pty=True) # Added get_pty=True for sudo
264         stdin.write(pw + "\n")
265         stdin.flush()
266
267         # Add a small delay and check stderr for sudo password errors
268         time.sleep(0.5)
269         if stderr.channel.recv_stderr_ready():
270             err_output = stderr.channel.recv_stderr(1024).decode(errors="ignore")
271             if "try again" in err_output.lower() or "incorrect password" in err_output.lower():
272                 q.put(f"!!!ERROR!! Sudo password incorrect or other sudo issue for Snort: {err_output}")
273                 return
274             # Log other stderr output as info, as Snort can be verbose on stderr
275             # q.put(f"!!!INFO!! Snort stderr: {err_output}")
276
277         for line in stdout:
278             q.put(line.strip())
279     except Exception as e:
280         # Ensure a serializable error message is put on the queue
281         try:
282             err_json = json.dumps({"error": str(e)})
283             q.put(f"!!!ERROR!! {err_json}")
284         except TypeError: # Fallback if error object itself is not serializable
285             q.put(f"!!!ERROR!! Error during Snort streaming: {str(e)}")
286     finally:
287         if client is not None:
288             client.close()
289
```

common.py ssh_stream_snort function

MITRE LookUp & Mitigations query

Functions to interact with MITRE ATT&CK enterprise .json locally.

```
lookup_and_mitigation.py x
mitre_querying > lookup_and_mitigation.py > get_techniques
1 import json
2 import sys
3 from typing import List, Dict, Any
4
5 def load_attack_data(path: str) -> List[Dict[str, Any]]:
6     with open(path, "r", encoding="utf-8") as f:
7         data = json.load(f)
8     return data.get("objects", [])
9
10 def index_attack_data(objects: List[Dict[str, Any]]) -> Dict[str, Dict[str, Any]]:
11     return {obj.get("id"): obj for obj in objects if "id" in obj}
12
13 def get_techniques(attack_objects: List[Dict[str, Any]], technique_ids: List[str]) -> Dict[str, Dict[str, Any]]:
14     result = {}
15     for obj in attack_objects:
16         if obj.get("type") != "attack-pattern":
17             continue
18         for ref in obj.get("external_references", []):
19             if ref.get("source_name") == "mitre-attack" and ref.get("external_id") in technique_ids:
20                 result[obj["id"]] = {
21                     "id": ref["external_id"],
22                     "name": obj.get("name"),
23                     "description": obj.get("description", "").strip(),
24                     "platforms": obj.get("x_mitre_platforms", []),
25                     "tactics": [phase["phase_name"] for phase in obj.get("kill_chain_phases", [])],
26                     "url": ref.get("url"),
27                     "stix_id": obj["id"],
28                 }
29     return result
30
31 def get_mitigations(attack_objects: List[Dict[str, Any]], techniques: Dict[str, Dict[str, Any]]) -> Dict[str, List[Dict[str, str]]]:
32     mitigation_map = {}
33     id_map = index_attack_data(attack_objects)
34
35     for obj in attack_objects:
36         if obj.get("type") != "relationship" or obj.get("relationship_type") != "mitigates":
37             continue
38         target_ref = obj.get("target_ref")
39         source_ref = obj.get("source_ref")
40         if target_ref in techniques:
41             mitigation_obj = id_map.get(source_ref)
42             if mitigation_obj and mitigation_obj.get("type") == "course-of-action":
43                 mitigation = {
44                     "name": mitigation_obj.get("name"),
45                     "description": mitigation_obj.get("description", "").strip()
46                 }
47                 mitigation_map.setdefault(target_ref, []).append(mitigation)
48
49     return mitigation_map
```

Lookup_and_mitigation.py file

SSH tunnel to enable Caldera locally

```
red_blue_emulator.py 3 x
pages > red_blue_emulator.py > ...
1 import streamlit as st
2 from ssh_tunnel import SSHTunnelForwarder
3 import paramiko
4 import os
5 # 36K to chat, 36K to generate
6 # --- Configuration & State ---
7 if 'ssh_tunnel_active' not in st.session_state:
8     st.session_state.ssh_tunnel_active = False
9 if 'ssh_server' not in st.session_state:
10     st.session_state.ssh_server = None
11
12 # --- Helper Functions ---
13 def start_ssh_tunnel_password(ssh_host, ssh_port, ssh_user, ssh_password, remote_bind_address, local_bind_address):
14     """Starts the SSH tunnel using password authentication."""
15     try:
16         server = SSHTunnelForwarder(
17             (ssh_host, int(ssh_port)),
18             ssh_username=ssh_user,
19             ssh_password=ssh_password,
20             remote_bind_address=remote_bind_address,
21             local_bind_address=local_bind_address,
22             # logger=create_logger(loglevel=1) # Optional: for debugging
23         )
24         server.start()
25         st.session_state.ssh_tunnel_active = True
26         st.session_state.ssh_server = server
27         st.success(f"SSH tunnel established to {ssh_host}!")
28         st.rerun() # Rerun to update iframe src
29         return server
30     except Exception as e:
31         st.error(f"Failed to start SSH tunnel: {e}")
32         if st.session_state.ssh_server:
33             st.session_state.ssh_server.stop()
34         st.session_state.ssh_tunnel_active = False
35         st.session_state.ssh_server = None
36         return None
37
38 def stop_ssh_tunnel():
39     """Stops the SSH tunnel."""
40     if st.session_state.ssh_server:
41         try:
42             st.session_state.ssh_server.stop()
43             st.session_state.ssh_tunnel_active = False
44             st.session_state.ssh_server = None
45             st.success("SSH tunnel stopped.")
46             st.rerun()
47         except Exception as e:
48             st.error(f"Error stopping tunnel: {e}")
49     else:
50         st.info("No active tunnel to stop.")
51
```

Red_blue_emulator.py start and stop ssh tunnel functions

APT group MITRE techniques ingestion with Kill Chain mapping

The code allows to request all techniques for specific APT groups and save them in JSON. As an experiment techniques are mapped to cyber kill chain stages and then to universal matrix based taxonomy. Number of corresponding stages of the universal matrix killchain (UM UKC) is added into JSON data. Not included in the public repo for reasons. Available upon request. Experimental UM based cyberattack taxonomy is described in referenced document [6]. This work was tied to initial project proposal [1].

```
1 # um_ukc_ingest.py
2 """Phase-2 helper library for VINO-AI (UM-UKC ingestion).
3
4 Version 1.0 2025-04-22
5 * Adds **command-line flags** (argparse) so you can:
6 * '--group APT29' - choose threat actor (default APT29)
7 * '--output apt29.json' - save mapped JSON to a file
8 * '--dedup' - remove duplicate ATT&CK IDs (parent+subtech duplicates)
9 * '--limit N' - print only first *N* rows to stdout (for preview)
10
11 Running example:
12 """
13 python3 um_ukc_ingest.py --group "APT29" --output "apt29_ukc.json" --dedup
14 """
15 The file 'apt29_ukc.json' is now ready for Phase-3 (Caldera build).
16 """
17 from __future__ import annotations
18
19 import argparse
20 import json
21 import sys
22 from datetime import datetime
23 from pathlib import Path
24 from typing import Any, Dict, List
25
26 try:
27     from mitreattack.stix20 import MitreAttackData
28 except ImportError as exc: # pragma: no cover
29     sys.exit("[FATAL] mitreattack-python is not installed: pip install mitreattack-python")
30
31 # -----
32 # Static mapping tables (excerpt)
33 # -----
34
```

um_ukc_ingest.py script header

Caldera APT auto launcher

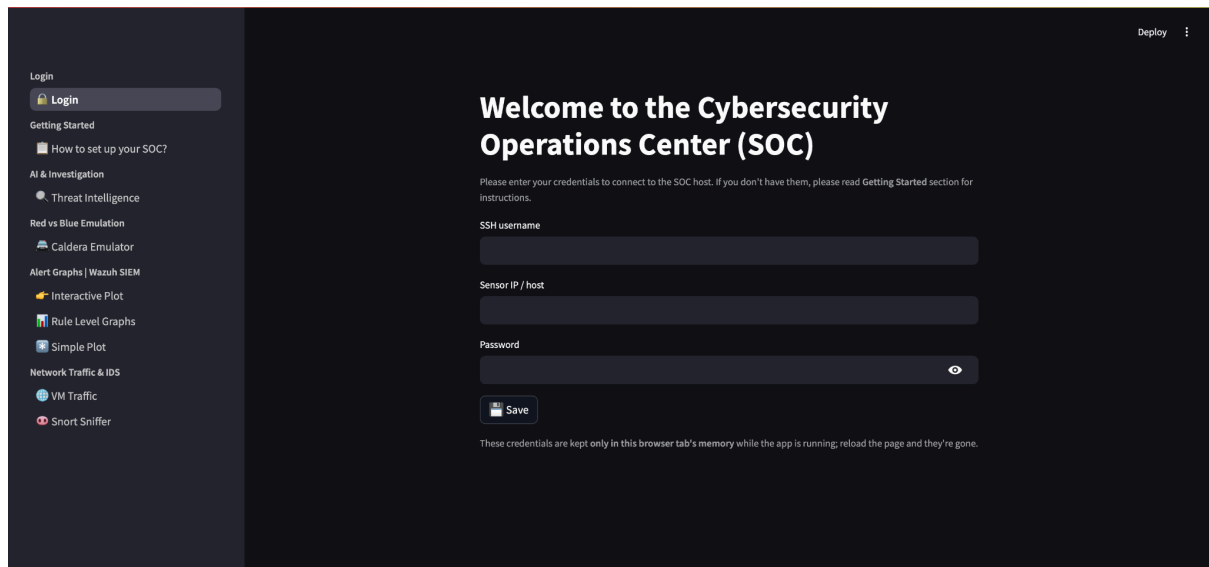
This script uses Caldera resetAPI to create and deploy an agent, create an operation and fill it with abilities corresponding to MITRE techniques for requested APT groups (by um_ukc_ingest.py). Also not included in the public repo for reasons. Available upon request.

```
19 # --- Caldera mini-SDK ---
20
21 class Caldera:
22     def __init__(self, root: str, key: str | None = None):
23         self.base = root.rstrip('/') + '/api/v2/'
24         self.h = {'accept': 'application/json',
25                  'Content-Type': 'application/json',
26                  **({'KEY': key} if key else {})}
27
28     def get(self, p: str, j: Dict | None = None, headers: Dict | None = None, timeout: int = 30, verify: bool = False):
29         r = requests.get(self.base + p, headers=self.h, json=j, timeout=timeout, verify=verify)
30         r.raise_for_status()
31         return r.json()
32
33     def post(self, p: str, j: Dict | None = None, headers: Dict | None = None, timeout: int = 30, verify: bool = False):
34         r = requests.post(self.base + p, headers=self.h, json=j, timeout=timeout, verify=verify)
35         r.raise_for_status()
36         return r.json()
37
38     abilities = lambda s: s.get('/abilities')
39     agents = lambda s: s.get('/agents')
40     objectives = lambda s: s.get('/objectives')
41     add_adv = lambda s, j: s.post('/adversaries', j)
42     add_op = lambda s, j: s.post('/operations', j)
43
44 # --- ATT&CK helpers (unchanged) ---
45
46 def attack_ids(ab: Dict) -> List[str]:
47     out = list(ab.get('attack_ids', []))
48     for k in ('attack_id', 'technique_id', 'subtechnique'):
49         if ab.get(k): out.append(ab[k])
50     t = ab.get('technique') or {}
51     if isinstance(t, dict):
52         out += [x for x in (t.get('attack_id'), t.get('subtechnique')) if x]
53     return out
54
55 def plat_ok(ab: Dict, want: str | None):
56     if not want: return True
57     want = want.lower()
58     p = ab.get('platforms')
59     if isinstance(p, dict) and want in (k.lower() for k in p): return True
60     if isinstance(p, list) and want in [x.lower() for x in p]: return True
61     return isinstance(ab.get('platform'), str) and ab['platform'].lower() == want
62
```

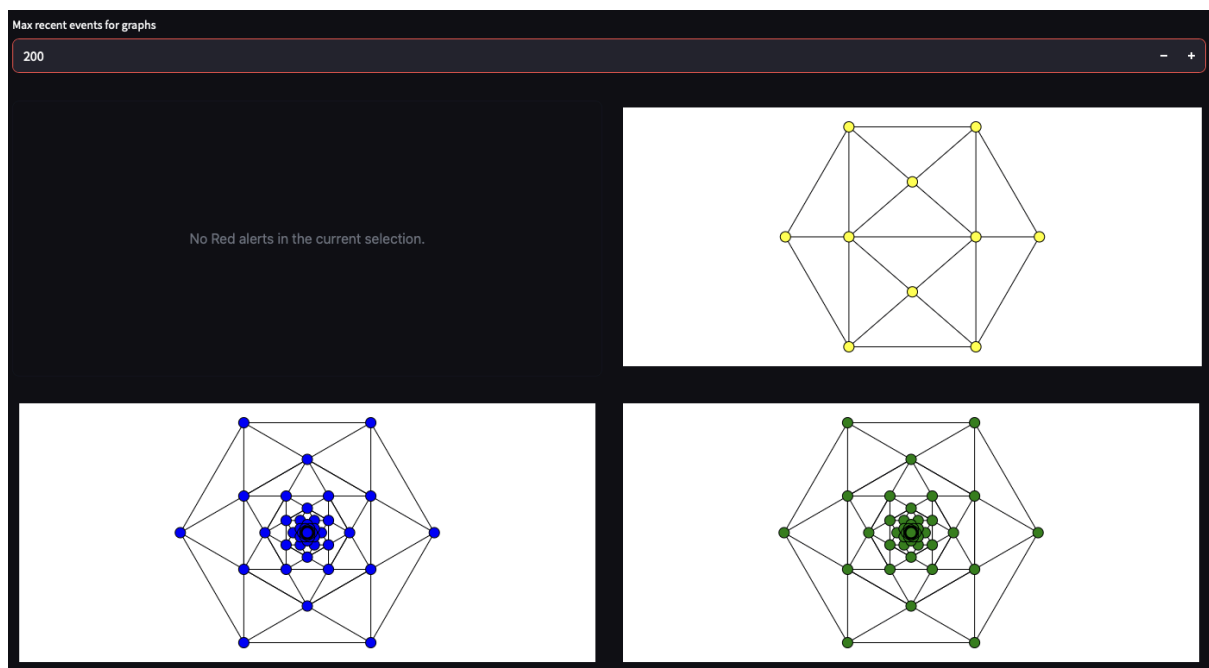
Um_ukc_caldera_builder_v3.py file, snippet of Caldera class

3.3 User Interface

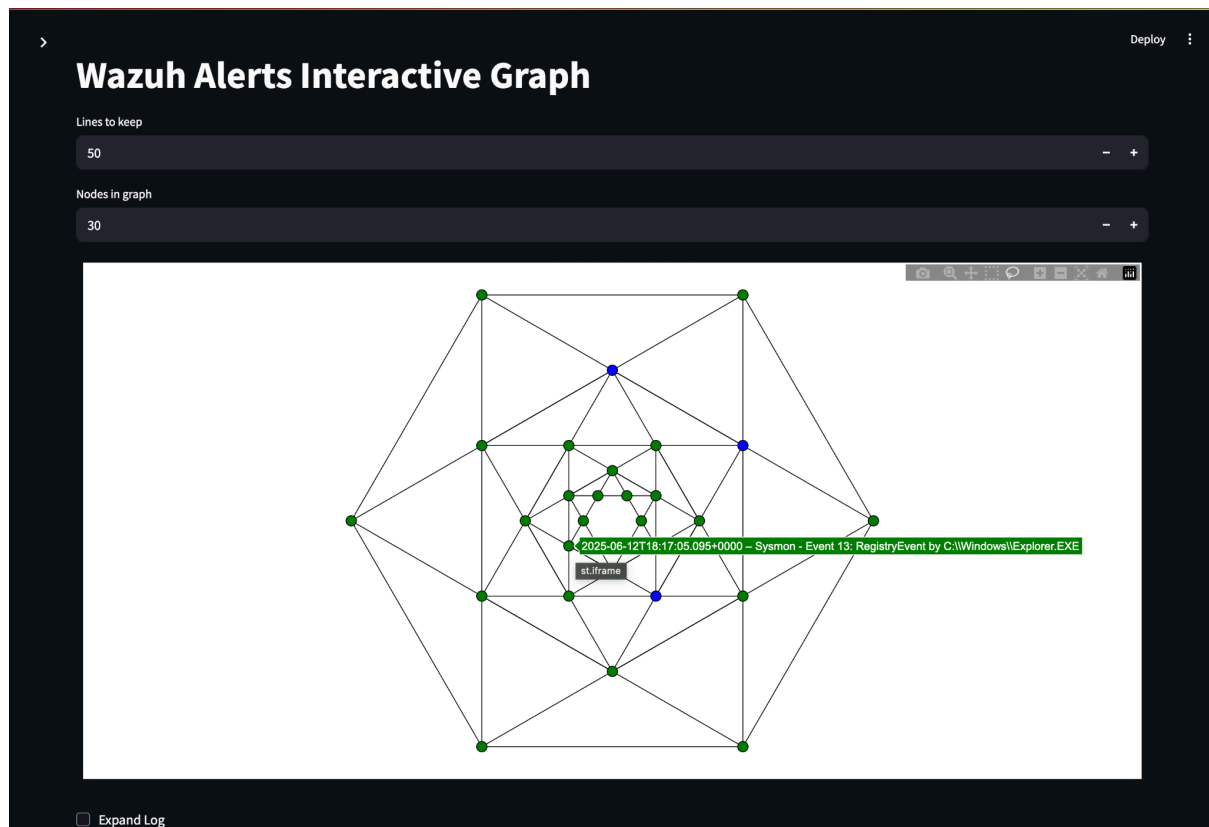
Several key pages presented. For more details about UI **watch the demo** in section 1 of this document.



Login page and left vertical menu allowing access to all other pages



Colored graphs, coloring is based on rule levels for alerts



Interactive graph, displaying node data on hover

4. Configuration and Change Management

Configuration files for all subsystems and components are stored in github with extensive readme coverage. Together with Getting started and installation guides for this application, soc_config github provides necessary files and simplifies manual editing (adding rules, configuring agents, etc.)

Github: https://github.com/Krasnomakov/soc_config

4.1 Infrastructure Management

The system uses multiple deployment scenarios:

- Docker-based deployment for Wazuh components
- Native installation for Snort IDS
- Platform-specific optimizations (especially for macOS)
- Powershell based deployment of Caldera agents on Windows target
- .exe installation of Wazuh agent on Windows target

4.2 Configuration Management

- Credential management through secure session state (Cisco VPN)
- SSH-based connectivity for remote monitoring
- Configurable alert rules and thresholds

- Custom rule support for Wazuh (test in getting started guide, rule example and integration guide in soc_config github)

4.3 Release Management

- Version-controlled codebase (github)
- Platform-specific optimizations (um_gen_metal.py for Apple Silicon, um_gen.py for other systems)
- Dependency management through requirements.txt

Conclusion

This realization report demonstrates that the SOC MVP project is a scalable software system that correlates with existing security frameworks while maintaining a modular and extensible architecture. The system's configuration and change management capabilities are structured, simplifying maintenance and updates.

Reference

1. Kraskov A. (2025). Initial Project Proposal
Canvas:
https://fhict.instructure.com/courses/13084/assignments/260587?module_item_id=1313006
2. Kraskov A. (2025). SOC MVP Infrastructure documentation
Canvas:
https://fhict.instructure.com/courses/13084/assignments/261913?module_item_id=1322378
3. Kraskov A. (2025). System Configuration Advice & Software Architecture Design
Canvas:
https://fhict.instructure.com/courses/13084/assignments/259595?module_item_id=1307686
4. Kraskov A. (2025). Technical report: Intuitive and visual decision-making support for SOC analysts
PDF:  Intuitive and visual decision-making support for SOC analysts .pdf
5. Sylvania, Shallwin. (2025). The Universal Matrix Visualizer - From logic to Computer Algorithm. 10.13140/RG.2.2.22510.11842.
6. Kraskov A. (2025). The Universal Matrix – Unified Kill Chain (UM-UKC) Taxonomy
PDF:  The Universal Matrix – Unified Kill Chain (UM-UKC) Taxonomy.pdf